Comparison Study of Robotic Middleware for Robotic Applications

Gergely Magyar[1], Peter Sinčák[1], and Zoltán Krizsán[2]

[1] Center for Intelligent Technologies, Department of Cybernetics and Artificial Intelligence, Technical University of Košice
gergely.magyar6@gmail.com, peter.sincak@tuke.sk
[2] Department of Information Technology, University of Miskolc
krizsan@iit.uni-miskolc.hu

Abstract. Developing a robot system is a hard task and it requires a special programming knowledge. Moreover, the development and the operation of every robot system needs common tasks such as state management, communication among parts, synchronization, etc. Several software platforms were introduced for supporting these, hence the researcher and developer can concentrate the novel ideas. This document gives an overview of various robotic middleware. Strictly defined, robotic middleware serves for realizing the communication between various software components. In a wider sense, they are helping the process of development of robotic applications. The document contains descriptions of systems such as Robot Operating System (ROS), RT-Middleware, OPRoS and Orocos. The report also contains a comparison of the above-listed.

1 Introduction

There are a lot of manufacturers offering various robotic hardware in the robotic market in recent years. These firms offer different software for their products and make it difficult for developers to create their own applications because they are supposed to use different tools, programming languages, etc. The idea of robotic middleware was introduced to solve this global problem. The robot middleware is a software framework containing tools and libraries that alleviate the development process of robot systems. Generally, the robot middleware has some tools for generating the source code of components and it is glue code that establishes the connection among components. Many projects came up with different architectures, but the component based model was common. It means that robotic applications would look like a complex system built from blocks called components. This architecture has many benefits [1]:

- Reusability – the same component can be used in various robotic systems. When developers need a component, they do not need to implement it from scratch, they can use one which already exists.

© Springer International Publishing Switzerland 2015

P. Sinčák et al. (eds.), *Emergent Trends in Robotics and Intelligent Systems*,
Advances in Intelligent Systems and Computing 316, DOI: 10.1007/978-3-319-10783-7_13

- Editability – the components can be easily edited and then used for different tasks.
- Facilitation of development – when testing algorithms, developers use multiple components. If they want to change a parameter or a part of a system, it is enough to change the parameters of some components. This way they can save time and be more efficient.

The above-listed robotic middleware can be used for creating components using various algorithms for artificial intelligence. These components can be used not only in research areas but also in non-robotic fields, for example, home appliances (such a system was created using RT-Middleware). The application of this idea can create a new section of the robotic market. Besides manufacturers of hardware and software, creators of components can play a significant role.

2 RT-Middleware Concept for Robotics

One of the most popular robotic middleware is RT-Middleware, which was created in Japan in 2002 as a project for NEDO (New Energy and Industrial Technology Development Organization). The project's original name was Robot Challenge Program. Its main purpose was to establish essential technologies for easy integration of robot systems having advanced functions of modularized software components [2].

As it was mentioned before, systems based on RT-Middleware consist of components – RT-components or simply RTCs. These components can be written in different programming languages (C++, JAVA, Python, .NET), run on different computers and communicate with each other via CORBA (Common Object Request Broker Architecture).

As shown on Fig. 1, an RT-component has a key part (Activity) which performs different tasks over data. The data processing is realized by input and output ports [3]. The main activity of the component is implemented via state machine, so the developer has to map the steps of the algorithm into these states: BORN, INITIALIZE, READY, STARTING, ACTIVE, STOPPING, ERROR, ABORTING, FATAL ERROR, EXITING, UNKNOWN [4].

The leading developer tool for creating applications in RT-Middleware is OpenRTM-aist, which is freely available. It has two main tools [5]:

- RTC Builder – serves for creating new RT-components through generating the source code
- RT System Editor – serves for connecting RT-components into an RT-System and controlling the runtime.

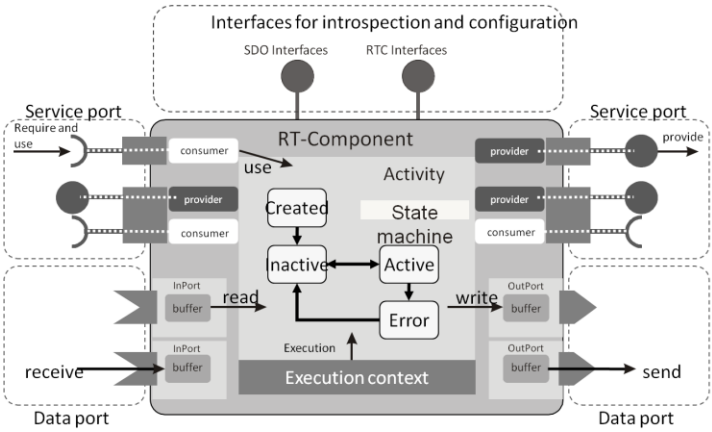


Fig. 1. Architecture of an RT-component [5]

OpenRTM-aist supports three programming languages: Python, C++ and Java. Besides OpenRTM-aist, some other implementations exist for the RT-Middleware technology, such as OpenRTM.NET, which uses Microsoft's .NET framework and has support for Visual Basic, C#, F# and C++.

Nowadays, there are over 300 available RT-components. Among these components, we can find RTCs for humanoids (Choromet, HRP) and various tasks using artificial intelligence, such as navigation, face recognition, speech processing.

3 ROS Concept for Robotics

Maybe the most popular robotic middleware is ROS, the Robot Operating System. This project is the result of collaboration between Stanford University and Willow Garage, and it was part of the STAIR project. The goals of this system are the following [6]:

- Peer-to-peer
- Tools-based
- Multi-lingual
- Thin
- Free and open-source

The main parts of systems based on ROS are nodes, messages, topics and services. Nodes are processes which are performing various tasks. These nodes are forming complex systems. In this context, we can describe these nodes as components, like RT-components. Nodes are communicating with each other via messages. Messages are data structures which consist of primitive data types, like integer, float, Boolean and others. Messages can be represented as arrays of primitive types or sets of other messages. Nodes send their messages by assigning them to topics, represented as strings. Topics can receive messages from more nodes, which can also send their

messages to more topics. This architecture is efficient but cannot handle synchronous processes. The concept of services was introduced to solve this problem. These services are represented by a string and a pair of messages. One is responsible for the request and the other for the response. Here, we can find a similarity with web services. On the other hand, services differ from topics. They can work with just one node and nodes can send their messages to just one service [6].

As it was mentioned above, ROS is open-source and freely available from the project's official website for the operating system Ubuntu. Other operating systems, such as Mac OS X and Windows, have just an experimental version [7]. Developers can also choose from several programming languages like C/C++, Python, Octave or Lisp. The communication between components written in different languages is ensured via IDL (interface definition language), which was created as part of the ROS project.

Nowadays, ROS has more than 2000 available components. Among these, we can find many libraries and ready-to-use components. Robot Operating System has also many components for robots, like Aldebaran's NAO, Willow Garage PR2, Lego NXT, iRobot Roomba, Qbo and others [7].

4 OPRoS Concept for Robotics

OPRoS (Open Software Platform for Robotic Services) serves for creating software modules from low level controlling to controlling complex systems. This middleware has its own interface which allows cooperation between components.

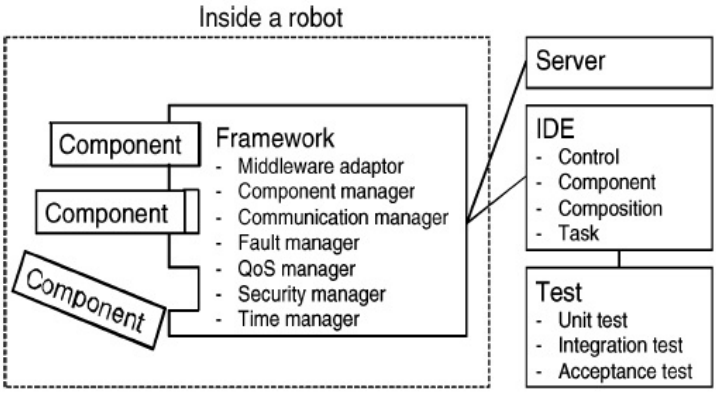


Fig. 2. Architecture of an OPRoS system [8]

As shown in Fig. 2, the robot contains a framework which serves for executing and managing components. This framework executes the components periodically or aperiodically, processes activities and errors of the components. It also contains various managers which ensure various functions of the components [8].

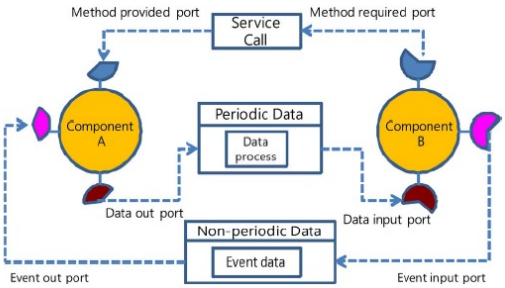


Fig. 3. A component in OPRoS [8]

As we can see in Fig. 3, a component in OPRoS, unlike components in RT-Middleware, has three types of ports: data ports, service ports and event ports. Periodic and aperiodic method calls are processed by data and service ports. Components can have more ports of the same type which are communicating with other ports via connectors [8].

The activities of the components are, like in RT-Middleware, realized by a state machine. However, in OPRoS it is simpler than in RTM. This structure is useful for monitoring and debugging the components. It has six states: CREATED, READY, ACTIVE, INACTIVE, ERROR, DESTROYED [8].

Users of the OPRoS platform have access to a server which contains already existing components or users can add their own components. In some cases, this server can be used for direct cooperation with robots. For example, we can execute the OPRoS components on the server and the robots just communicate with them. In this case, the robots are acting as clients. The significant benefit of this feature is that robots with low hardware specifications can also use the OPRoS platform.

Since OPRoS is a relatively young project, it does not have as many available components as RT-Middleware or ROS. Among supported robots, we can mention iRobiQ or FURo [8].

5 Orocos Concept for Robotics

Another interesting project for making the development of robotic applications easier is Orocos (Open Robot Control Software). Its primary purpose is to create controlling modules for robots with the following attributes [9]:

- Open-source – source codes are freely available for use and editing
- Modular and flexible – chance to change just parts of the whole system
- High quality – modules have high quality from the technical view, have a comprehensive documentation and as software are robust
- Available for all computer platforms (Linux, Mac OS, Windows)
- Localized for all languages

Orocos uses the divide and conquer method for the development of robotic systems. These are containing libraries and modules. Modules can be [9]:

- Supporting modules – software without robotic elements but needed for a working robotic system. As an example, we can provide simulators, 3D visualization or tools for generating documentation. These modules can be parts of other open-source projects.
- Robotic modules – are specific robotic algorithms implemented as modules, for example, motion planners for mobile robots, kinematics and dynamics of systems, etc. These modules cooperate with supporting modules.
- Components – are CORBA objects needed for creating robotic applications (Fig. 4).

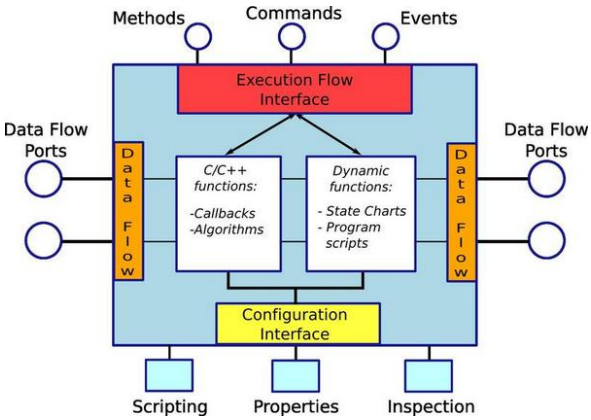


Fig. 4. A component in Orocos [9]

For creating components based on the Orocos platform, we need Orocos Toolchain, which is freely available on the project's official site. Toolchain is available on various distributions of Linux (Fedora, Debian, Ubuntu), Mac OS X and Windows (cooperation with Visual Studio). It contains extensions for systems such as ROS, Rock and Yarp. The Toolchain consists of the following parts [10]:

- AutoProj – tool for downloading and compilation of libraries
- Real-Time Toolkit – framework needed for creating components in C++ programming language
- Orocos Component Library – set of components needed for working with the application
- OroGen and TypeGen – tools for generating source codes of components

Among the applications of this framework, we can mention motion specification for robots, software components for an autonomous car, 3D motion tracking or dynamic parameter identification of a robot and others.

6 Comparison of Mentioned Robotic Middleware

On the previous pages, we described four robotic middleware, namely RT-Middleware, ROS, OPRoS and Orocos. These platforms can be compared in many ways which are important both from the view of the user and the developer. As the main criteria of the user, we considered the price of the systems, whether they are freely available and what operating systems are supported. From the point of view of the developer, a graphical IDE for managing systems and a simulator for experimenting with the developed systems are important. Table 1 also contains other criteria such as the composite component, which means that we can create one big component containing smaller components. Another important attribute of these systems is independence from the robot software architecture. It means that we can apply the components for robots and it doesn't matter what kind of software is added to them. The real-time criteria describe if we can change the parameters of the components during the run

Table 1. Comparison of robotic middleware

	RT-Middleware	ROS	OPRoS	Orocos
Open-source	yes	yes	yes	yes
Windows	yes	no	yes	yes
Linux	yes	yes	yes	yes
Composite component	yes	no	yes	no
Robot software architecture independent	yes	yes	yes	yes
Graphical IDE	yes	no	yes	no
Simulation environment	yes – OpenHRP, Choreonoid	yes – Stage, RVIZ	yes – OPRoS Simulator	no
Real-time	yes	no	planned	yes

As we can learn from Table 1, all robotic middleware have their advantages and disadvantages. Based on this table, RT-Middleware can be considered as the perfect middleware, but it has fewer components than ROS and does not support as many robots as ROS. A big weakness of ROS can be the fact that Windows is not among the supported operating systems and it does not have a graphical IDE. The developers usually like to have a simulation environment for experimenting with the components. From this point of view Orocos, is not a smart choice, on the other hand, it has a lot of useful libraries.

7 Conclusion

In this paper, we focused on describing four robotic middleware which can be used for creating robotic applications. These were RT-Middleware, ROS, OPRoS and Orocos. In Chapter 6, using a table, we compared them through various attributes. Based on this table, we can choose a platform which is the best for various purposes. Besides the above-mentioned platforms, there are many middleware which can be used for the same tasks, such as MIRO, Player/Stage, URBI, ORiN, Orca and others.

Acknowledgments. This research was supported by the Hungarian National Scientific Research Fund grant no: OTKA K77809.

References

1. Ando, N., et al.: RT(Robot Technology)-Component and its Standardization. In: International Joint Conference (2006)
2. Ando, N., et al.: RT-Component Object Model in RT-Middleware – Distributed Component Middleware for RT (Robot Technology). In: International Symposium on Computational Intelligence in Robotics and Automation (2005)
3. Ando, N., et al.: RT-Middleware: Distributed Component Middleware for RT (Robot Technology). In: International Conference on Intelligent Robots and Systems, pp. 3933–3938 (2005)
4. Ando, N., et al.: Composite Component Framework for RT-Middleware (Robot Technology Middleware). In: International Conference on Advanced Intelligent Mechatronics, pp. 1330–1335 (2005)
5. OpenRTM-aist, `http://www.openrtm.org`
6. Quigley, M., et al.: ROS: An open-source Robot Operating System. In: ICRA Workshop on Open Source Software (2009)
7. ROS Wiki, `http://www.ros.org/wiki`.
8. Soohee, H.A.N., et al.: Open Software Platform for Robotic Services. In: Transactions on Automation Science and Engineering (2012)
9. Bruyninckx, H.: Open Robot Control Software: The OROCOS Project. In: International Conference on Robotics and Automation, pp. 2523–2528 (2001)
10. The Orocos Project, `http://www.orocos.org/`